MANUAL TÉCNICO — AGROGESTIÓN

Índice

1. Introducción
2. Arquitectura de la aplicación
   - 2.1. Frontend
   - 2.2. Backend
3. Documentación técnica
   - 3.1. Análisis
   - 3.2. Desarrollo (Diagramas de secuencia)
   - 3.3. Pruebas realizadas
4. Proceso de despliegue
5. Propuestas de mejoras
6. Bibliografía

---

1. Introducción

Agrogestión es una aplicación web desarrollada como proyecto final del ciclo formativo de Grado Superior en Desarrollo de Aplicaciones Web (DAW) en el IES Albarregas de Mérida.

El objetivo del proyecto es crear una herramienta web que permita digitalizar la gestión del trabajo en explotaciones agrícolas, facilitando la coordinación entre los distintos roles que intervienen en la actividad: administradores, gerentes, capataces y trabajadores.

La aplicación se ha desarrollado siguiendo una arquitectura moderna de tipo **SPA (Single Page Application)** con un frontend en React y un backend gestionado íntegramente por Supabase (Backend as a Service).

Objetivos del proyecto

- Permitir el registro y autenticación de usuarios con selección de rol.
- Implementar un sistema de roles jerárquico (ADMIN, GERENTE, CAPATAZ, TRABAJADOR).
- Gestionar terrenos agrícolas (alta, baja lógica, edición).
- Crear, asignar y hacer seguimiento de tareas agrícolas.
- Establecer relaciones entre gerentes-capataces y capataces-trabajadores.
- Garantizar la seguridad de los datos mediante políticas RLS (Row Level Security).
- Ofrecer una interfaz responsive accesible desde cualquier dispositivo (escritorio, tablet y móvil).
- Permitir la edición de perfil personal por parte de cada usuario.
- Proporcionar estadísticas visuales con tarjetas de resumen y gráficos.
- Soportar internacionalización en tres idiomas: español, inglés y rumano.

---

2. Arquitectura de la aplicación

Agrogestión utiliza una arquitectura **cliente-servidor desacoplada**:

```
┌──────────────────────┐          HTTPS/REST          ┌──────────────────────────┐
│                      │  ◄──────────────────────►    │                          │
│   FRONTEND (SPA)     │                              │   BACKEND (Supabase)     │
│   React + Vite       │                              │   PostgreSQL + Auth      │
│   TailwindCSS        │                              │   RLS + API REST         │
```

```
│                    │                              │                          │
│   Navegador web    │                              │   Servidor en la nube    │
└────────────────────┘                              └──────────────────────────┘
```

- **Frontend:** Aplicación SPA que se ejecuta en el navegador del usuario.
- **Backend:** Supabase proporciona base de datos PostgreSQL, autenticación, API REST auto-generada y políticas de seguridad.
- **Comunicación:** El frontend se comunica con Supabase a través de la librería `@supabase/supabase-js` mediante llamadas HTTPS.

2.1. Frontend

2.1.1. Tecnologías usadas

Tecnología	Versión	Descripción
React	19.2.0	Librería para construir interfaces de usuario declarativas.
TypeScript	5.9.3	Superset de JavaScript con tipado estático.
Vite	7.2.4	Herramienta de build ultrarrápida para proyectos frontend.
TailwindCSS	4.1.18	Framework de utilidades CSS para diseño responsive.
@vitejs/plugin-react-swc	4.2.2	Plugin de Vite que usa SWC para compilar React (más rápido).
React Router	7.13.1	Librería de enrutamiento para navegación SPA.
i18next / react-i18next	25.8.14 / 16.5.4	Librería de internacionalización (i18n) para soporte multi-idioma.
Zustand	5.0.11	Gestión de estado global ligera y sencilla.
dotenv	17.2.3	Gestión de variables de entorno.
ESLint	9.39.1	Herramienta de análisis estático para detectar errores en JS/TS.
typescript-eslint	8.46.4	Plugin de ESLint para soporte de TypeScript.

2.1.2. Entorno de desarrollo

Componente	Detalle
Sistema operativo	Windows 10/11
IDE	Visual Studio Code
Runtime	Node.js (versión LTS recomendada: 20.x o superior)
Gestor de paquetes	npm (incluido con Node.js)
Control de versiones	Git + GitHub (repositorio: Juanfrina/Agrogestion.Web)
Navegador de pruebas	Google Chrome / Firefox / Edge

2.2. Backend

2.2.1. Tecnologías usadas

Tecnología	Descripción
Supabase	Backend as a Service (BaaS) basado en PostgreSQL.
PostgreSQL	Motor de base de datos relacional (gestionado por Supabase).
Supabase Auth	Sistema de autenticación integrado (email + contraseña).
Supabase API REST	API auto-generada a partir del esquema de la base de datos.
Row Level Security (RLS)	Políticas de seguridad a nivel de fila en PostgreSQL.
@supabase/supabase-js	2.93.3 — Librería cliente para comunicar el frontend con Supabase.

2.2.2. Entorno de desarrollo

Componente	Detalle
Plataforma	Supabase Cloud (https://supabase.com)
Base de datos	PostgreSQL gestionado en la nube de Supabase
Panel de administración	Dashboard web de Supabase para configuración de tablas y RLS
Variables de entorno	VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY en archivo .env

---

3. Documentación técnica

3.1. Análisis

3.1.1. Requisitos funcionales

ID	Requisito	Prioridad
RF-01	El sistema debe permitir el registro de nuevos usuarios con selección de rol.	Alta
RF-02	El sistema debe permitir el inicio de sesión con autenticación segura.	Alta
RF-03	El sistema debe asignar un panel diferente a cada usuario según su rol.	Alta
RF-04	El administrador debe poder ver, editar y desactivar/reactivar usuarios.	Alta
RF-05	El administrador debe poder asignar y cambiar roles a los usuarios.	Alta
RF-06	El gerente debe poder gestionar terrenos (alta, baja lógica, edición).	Alta
RF-07	El gerente debe poder crear tareas agrícolas asociadas a terrenos.	Alta

RF-08	El gerente debe poder asignar tareas a capataces.	Alta
RF-09	El gerente debe poder consultar el estado de las tareas.	Alta
RF-10	El gerente debe poder asociar y desasociar capataces a su equipo.	Media
RF-11	El capataz debe poder ver las tareas que le han sido asignadas.	Alta
RF-12	El capataz debe poder cambiar el estado de una tarea (pendiente → asignada → aceptada → en progreso → completada).	Alta
RF-13	El capataz debe poder asignar trabajadores de apoyo a tareas concretas.	Media
RF-14	El capataz debe poder gestionar su equipo de trabajadores habituales.	Media
RF-15	El trabajador debe poder ver las tareas en las que participa.	Alta
RF-16	El trabajador debe poder consultar los detalles de cada tarea asignada.	Media
RF-17	El sistema debe aplicar filtros por terreno, estado y usuario en los listados.	Media
RF-18	El sistema debe incluir una landing page informativa con acceso a login y registro.	Baja
RF-19	El sistema debe permitir cerrar sesión de forma segura.	Alta
RF-20	Cada usuario solo debe poder acceder a los datos que le corresponden según su rol.	Alta
RF-21	Cada usuario debe poder editar su perfil personal (nombre, apellidos, email, teléfono, dirección).	Media
RF-22	El sistema debe permitir la recuperación de contraseña por email.	Alta
RF-23	El administrador debe poder visualizar estadísticas globales con gráficos.	Media
RF-24	El sistema debe soportar internacionalización en español, inglés y rumano.	Media
RF-25	La interfaz debe ser responsive con menú hamburguesa en dispositivos móviles.	Media
RF-26	Los usuarios involucrados en una tarea deben poder añadir y ver comentarios, con indicador visual 💬 del número de comentarios.	Media

RF-27	El sistema debe generar notificaciones automáticas al asignar tareas o trabajadores.	Media
RF-28	El trabajador debe poder aceptar o rechazar su asignación a una tarea.	Media
RF-29	Los capataces y trabajadores deben ver una campanita 🔔 con contador de notificaciones no leídas en la cabecera.	Media
RF-30	Los usuarios deben poder marcar notificaciones como leídas (individualmente o todas a la vez).	Media
RF-31	El sistema debe enviar un email automático al destinatario cada vez que se genera una notificación.	Media

3.1.2. Requisitos no funcionales

ID	Requisito	Categoría
RNF-01	La aplicación debe ser responsive y funcionar en dispositivos móviles, tablets y PCs.	Usabilidad
RNF-02	La interfaz debe ser intuitiva y no requerir formación previa.	Usabilidad
RNF-03	El tiempo de carga inicial no debe superar los 3 segundos con conexión estándar.	Rendimiento
RNF-04	Las consultas a base de datos deben responder en menos de 1 segundo.	Rendimiento
RNF-05	La autenticación debe usar tokens JWT seguros gestionados por Supabase Auth.	Seguridad
RNF-06	Las políticas RLS deben garantizar que ningún usuario acceda a datos no autorizados.	Seguridad
RNF-07	Las contraseñas deben almacenarse cifradas (bcrypt) en Supabase Auth.	Seguridad
RNF-08	La comunicación con el servidor debe realizarse siempre sobre HTTPS.	Seguridad
RNF-09	El código fuente debe seguir convenciones de estilo validadas con ESLint.	Mantenibilidad
RNF-10	La aplicación debe usar TypeScript para minimizar errores en tiempo de desarrollo.	Mantenibilidad
RNF-11	La estructura del proyecto debe ser modular y escalable.	Mantenibilidad
RNF-12	La aplicación debe ser compatible con Chrome, Firefox, Edge y Safari actualizados.	Compatibilidad

RNF-13	El sistema debe estar disponible 24/7 gracias al alojamiento en Supabase Cloud.	Disponibilidad

3.1.3. Modelo de datos (Base de datos)

La base de datos se implementa en PostgreSQL gestionado por Supabase. Se compone de **10 tablas** principales interrelacionadas.

Tablas del sistema:

Tabla	Descripción
rol	Tipos de usuario: ADMIN (1), GERENTE (2), CAPATAZ (3), TRABAJADOR (4).
perfiles	Perfil extendido del usuario (nombre, apellidos, dni, email, tlf, dirección, rol).
terreno	Terrenos gestionados por los gerentes.
estados_tarea	Flujo de estados: pendiente, asignada, aceptada, rechazada, en progreso, completada.
tarea	Tareas agrícolas vinculadas a terrenos, gerentes y capataces.
gerente_capataz	Relación N:M entre gerentes y capataces.
tarea_trabajador	Trabajadores asignados a tareas con estado de aceptación.
capataz_trabajador	Relación estable entre capataz y sus trabajadores habituales.
comentarios_tarea	Comentarios de usuarios en tareas concretas.
notificaciones	Notificaciones automáticas generadas por triggers (asignación, cambio de estado).

3.1.4. Diagrama Entidad-Relación

```
┌──────────┐       ┌──────────────┐       ┌──────────────────┐
│   ROL    │1────N │   PERFILES   │       │   ESTADOS_TAREA  │
│          │       │              │       │                  │
│id_rol(PK)│       │ id(PK)       │       │ id_estado(PK)    │
│ nombre   │       │ nombre       │       │ nombre           │
│ descrip. │       │ apellidos    │       │ descripcion      │
└──────────┘       │ dni          │       └──────────────────┘
                   │ email        │                │1
                   │ tlf          │                │
                   │ direccion    │                │
                   │ id_rol(FK)   │                │
                   │ fecha_baja   │                │
                   │ created_at   │                │
                   └──────────────┘                │
                      │  │  │                      │
          ┌───────────┘  │  └──────────┐           │
          │              │             │           │
     ┌────▼────┐    ┌────▼────┐   ┌────▼────┐      │
     │GERENTE  │    │CAPATAZ  │   │TRABAJADOR│     │
     └─────────┘    └─────────┘   └─────────┘      │
```

```
        │  │           │  │           │  │           │
        │  │N         M│  │           │  │           │
        │  └────┐   ┌────┘  │           │  │           │
        │       │   │       │           │  │           │
        │ ┌─────▼───▼────────┐         │  │           │
        │ │GERENTE_CAPATAZ   │         │  │           │
        │ │                  │         │  │           │
        │ │id_gerente(FK)    │         │  │           │
        │ │id_capataz(FK)    │         │  │           │
        │ │fecha_asignacion  │         │  │           │
        │ │fecha_baja        │         │  │           │
        │ └──────────────────┘         │  │           │
        │                   ┌──────────┘  │           │
        │                   │             │           │
        │             ┌─────▼─────────────▼────┐      │
        │             │  CAPATAZ_TRABAJADOR    │      │
        │             │                        │      │
        │             │ id_capataz(FK)         │      │
        │             │ id_trabajador(FK)      │      │
        │             │ fecha_asignacion       │      │
        │             │ fecha_baja             │      │
        │             └────────────────────────┘      │
        │                                             │
    ┌───▼──────────┐                                  │
    │   TERRENO    │                                  │
    │              │                                  │
    │ id_terreno(PK)│                                 │
    │ nombre       │                                  │
    │ ubicacion    │                                  │
    │ tipo_cultivo │                                  │
    │ estado       │                                  │
    │ id_gestor(FK)│                                  │
    │ fecha_baja   │                                  │
    └──────────────┘                                  │
        │1                                            │
        │                                             │
    ┌───▼──────────────────────┐                      │
    │         TAREA            │                      │
    │                          │──────────────────────┘N
    │ id_tarea(PK)             │
    │ nombre                   │
    │ descripcion              │
    │ fecha_inicio             │
    │ fecha_fin                │
    │ id_terreno(FK)→terreno   │
    │ id_gerente(FK)→perfiles  │
    │ id_capataz(FK)→perfiles  │
    │ id_estado(FK)→estados_tarea│
    │ fecha_baja               │
    │ created_at               │
    │ updated_at               │
    └──────────────────────────┘
      │         │
```

```
      │1       │1
      │        │
      │   ┌────▼──────────────────┐
      │   │  TAREA_TRABAJADOR     │
      │   │                       │
      │   │ id_tarea(FK)→tarea    │
      │   │ id_trabajador(FK)     │
      │   │ id_capataz_asigna(FK) │
      │   │ estado                │
      │   │ fecha_asignacion      │
      │   │ fecha_respuesta       │
      │   │ fecha_baja            │
      │   └───────────────────────┘
      │
      ├──────────────────────────┐
      │                          │
┌────▼──────────────────┐   ┌────▼──────────────────┐
│  COMENTARIOS_TAREA    │   │    NOTIFICACIONES     │
│                       │   │                       │
│ id_comentario(PK)     │   │ id_notificacion(PK)   │
│ id_tarea(FK)→tarea    │   │ id_destinatario(FK)   │
│ id_autor(FK)→perfiles │   │   →perfiles           │
│ contenido             │   │ tipo                  │
│ created_at            │   │ titulo                │
└───────────────────────┘   │ mensaje               │
                            │ id_tarea(FK)→tarea    │
                            │ leida                 │
                            │ created_at            │
                            └───────────────────────┘
```

3.1.5. Diccionario de datos

Tabla: rol

Campo	Tipo	Restricciones	Descripción
id_rol	INTEGER	PK, NOT NULL, CHECK (1,2,3,4)	Identificador único del rol.
nombre	VARCHAR(50)	NOT NULL, UNIQUE	Nombre del rol (ADMIN, GERENTE...).
descripcion	VARCHAR(255)		Descripción del rol.

Tabla: perfiles

Campo	Tipo	Restricciones	Descripción
id	UUID	PK, NOT NULL, FK → auth.users(id) ON DELETE CASCADE	Identificador único (vinculado a auth.users).
nombre	VARCHAR(100)	NOT NULL	Nombre del usuario.
apellidos	VARCHAR(100)	NOT NULL	Apellidos del usuario.

dni	VARCHAR(9)	UNIQUE, NOT NULL	Documento nacional de identidad.
email	VARCHAR(100)	UNIQUE, NOT NULL	Correo electrónico.
tlf	VARCHAR(15)	NOT NULL	Teléfono de contacto.
direccion	VARCHAR(255)	NOT NULL	Dirección postal.
id_rol	INTEGER	NOT NULL, DEFAULT 4, FK → rol(id_rol)	Rol asignado al usuario.
fecha_baja	DATE	DEFAULT NULL	Si no es null, el usuario está dado de baja.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de creación del registro.

Tabla: terreno

Campo	Tipo	Restricciones	Descripción
id_terreno	SERIAL	PK, NOT NULL	Identificador único del terreno.
nombre	VARCHAR(100)	NOT NULL	Nombre del terreno.
ubicacion	VARCHAR(255)	NOT NULL	Dirección o referencia geográfica.
tipo_cultivo	VARCHAR(100)	NOT NULL	Tipo de cultivo (olivo, viña, cereal...).
estado	VARCHAR(20)	NOT NULL, DEFAULT 'activo'	Estado actual del terreno.
id_gestor	UUID	NOT NULL, FK → perfiles(id)	Gerente que gestiona el terreno.
fecha_baja	DATE	DEFAULT NULL	Si no es null, el terreno está de baja.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de creación.

Tabla: estados_tarea

Campo	Tipo	Restricciones	Descripción
id_estado	SERIAL	PK, NOT NULL	Identificador del estado.
nombre	VARCHAR(50)	NOT NULL, UNIQUE	Nombre: pendiente, asignada, aceptada, rechazada, en progreso, completada.
descripcion	VARCHAR(255)		Descripción del estado.

Tabla: tarea

Campo	Tipo	Restricciones	Descripción
id_tarea	SERIAL	PK, NOT NULL	Identificador de la tarea.
nombre	VARCHAR(100)	NOT NULL	Nombre descriptivo de la tarea.

descripcion	TEXT		Descripción detallada de la tarea.
fecha_inicio	DATE	NOT NULL	Fecha prevista de inicio.
fecha_fin	DATE	NOT NULL, CHECK (≥ fecha_inicio)	Fecha prevista de finalización.
id_terreno	INTEGER	NOT NULL, FK → terreno(id_terreno)	Terreno donde se realiza.
id_gerente	UUID	NOT NULL, FK → perfiles(id)	Gerente que creó la tarea.
id_capataz	UUID	FK → perfiles(id)	Capataz asignado a ejecutar la tarea.
id_estado	INTEGER	DEFAULT 1, FK → estados_tarea(id_estado)	Estado actual de la tarea.
fecha_baja	DATE	DEFAULT NULL	Si no es null, la tarea está eliminada.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de creación.
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Última modificación.

Tabla: gerente_capataz

Campo	Tipo	Restricciones	Descripción
id_gerente	UUID	PK, FK → perfiles(id)	Gerente de la relación.
id_capataz	UUID	PK, FK → perfiles(id)	Capataz asociado al gerente.
fecha_asignacion	DATE	DEFAULT CURRENT_DATE	Fecha en que se creó la asociación.
fecha_baja	DATE	DEFAULT NULL	Si no es null, la relación está de baja.

Tabla: tarea_trabajador

Campo	Tipo	Restricciones	Descripción
id_tarea	INTEGER	PK, FK → tarea(id_tarea)	Tarea en la que participa.
id_trabajador	UUID	PK, FK → perfiles(id)	Trabajador asignado como apoyo.
id_capataz_asigna	UUID	NOT NULL, FK → perfiles(id)	Capataz que realizó la asignación.
estado	VARCHAR(20)	NOT NULL, DEFAULT 'PENDIENTE'	Estado: PENDIENTE, ACEPTADA, RECHAZADA, COMPLETADA.
fecha_asignacion	DATE	DEFAULT CURRENT_DATE	Fecha en que se asignó al trabajador.
fecha_respuesta	TIMESTAMPTZ		Fecha en que el trabajador aceptó/rechazó.
fecha_baja	DATE	DEFAULT NULL	Si no es null, la asignación está de baja.

Tabla: capataz_trabajador

Campo	Tipo	Restricciones	Descripción
id_capataz	UUID	PK, FK → perfiles(id)	Capataz de la relación.
id_trabajador	UUID	PK, FK → perfiles(id)	Trabajador habitual del capataz.
fecha_asignacion	DATE	DEFAULT CURRENT_DATE	Fecha en que se creó la asociación.
fecha_baja	DATE	DEFAULT NULL	Si no es null, la relación está de baja.

Tabla: comentarios_tarea

Campo	Tipo	Restricciones	Descripción
id_comentario	SERIAL	PK, NOT NULL	Identificador único del comentario.
id_tarea	INTEGER	NOT NULL, FK → tarea(id_tarea) ON DELETE CASCADE	Tarea a la que pertenece el comentario.
id_autor	UUID	NOT NULL, FK → perfiles(id)	Autor del comentario.
contenido	TEXT	NOT NULL	Texto del comentario.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de creación.

Tabla: notificaciones

Campo	Tipo	Restricciones	Descripción
id_notificacion	SERIAL	PK, NOT NULL	Identificador único de la notificación.
id_destinatario	UUID	NOT NULL, FK → perfiles(id)	Usuario destinatario.
tipo	VARCHAR(50)	NOT NULL, CHECK (tipos válidos)	Tipo: TAREA_ASIGNADA, TAREA_ACEPTADA, COMENTARIO_NUEVO...
titulo	VARCHAR(200)	NOT NULL	Título breve de la notificación.
mensaje	TEXT		Mensaje descriptivo (opcional).
id_tarea	INTEGER	FK → tarea(id_tarea) ON DELETE CASCADE	Tarea relacionada (opcional).
leida	BOOLEAN	DEFAULT FALSE	Indica si la notificación ha sido leída.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Fecha de creación.

3.1.6. Casos de uso

Actores del sistema:

Actor	Descripción
Visitante	Usuario no autenticado que accede a la landing page.
Admin	Administrador del sistema con control total.
Gerente	Gestiona terrenos, crea tareas y supervisa capataces.
Capataz	Ejecuta tareas, actualiza estados y coordina trabajadores.
Trabajador	Consulta las tareas en las que participa.

Casos de uso principales:

CU	Nombre	Actor(es)	Descripción
CU-01	Registrarse	Visitante	El visitante crea una cuenta con email y contraseña.
CU-02	Iniciar sesión	Todos	El usuario se autentica con email y contraseña.
CU-03	Cerrar sesión	Todos	El usuario cierra su sesión de forma segura.
CU-04	Ver panel según rol	Todos (autenticados)	El sistema redirige al panel correspondiente al rol.
CU-05	Gestionar usuarios	Admin	CRUD de usuarios y asignación de roles.
CU-06	Alta de terreno	Gerente	El gerente crea un nuevo terreno.
CU-07	Editar terreno	Gerente	El gerente modifica datos de un terreno.
CU-08	Baja lógica de terreno	Gerente	El gerente desactiva un terreno.
CU-09	Crear tarea	Gerente	El gerente crea una tarea asociada a un terreno.
CU-10	Asignar tarea a capataz	Gerente	El gerente asigna una tarea pendiente a un capataz.
CU-11	Ver estado de tareas	Gerente	El gerente consulta el estado de las tareas.
CU-12	Asociar capataz	Gerente	El gerente asocia un capataz a su equipo.
CU-13	Aceptar/Rechazar tarea	Capataz	El capataz acepta o rechaza una tarea asignada.

CU-14	Completar tarea	Capataz	El capataz marca una tarea como completada.
CU-15	Asignar trabajador a tarea	Capataz	El capataz asigna un trabajador de apoyo a una tarea.
CU-16	Gestionar trabajadores	Capataz	El capataz administra su equipo de trabajadores habituales.
CU-17	Ver mis tareas asignadas	Trabajador	El trabajador consulta las tareas en las que participa.
CU-18	Filtrar tareas	Gerente, Capataz	Filtrado de tareas por terreno, estado o usuario.
CU-19	Editar perfil personal	Todos (autenticados)	El usuario modifica su nombre, apellidos, teléfono y dirección.
CU-20	Recuperar contraseña	Visitante	Solicita restablecimiento de contraseña por email.
CU-21	Admin: editar datos de usuario	Admin	El admin edita nombre, apellidos, email, tlf, dirección y rol de cualquier usuario mediante un modal.
CU-22	Seleccionar rol en registro	Visitante	Durante el registro, el usuario elige su rol (Gerente, Capataz o Trabajador).
CU-23	Cambiar idioma	Todos	El usuario cambia el idioma de la interfaz (ES / EN / RO).
CU-24	Añadir comentario a tarea	Gerente, Capataz, Trabajador	El usuario deja un comentario en una tarea en la que participa.
CU-25	Aceptar/Rechazar asignación	Trabajador	El trabajador acepta o rechaza su asignación a una tarea.
CU-26	Ver notificaciones	Capataz, Trabajador	El usuario pulsa la campanita 🔔 en la cabecera para ver sus notificaciones con contador de no leídas.
CU-27	Marcar notificaciones leídas	Capataz, Trabajador	El usuario marca notificaciones como leídas (individualmente o todas a la vez).
CU-28	Recibir email de notificación	Capataz, Trabajador	El sistema envía automáticamente un email al destinatario cada vez que se crea una notificación (vía Brevo + pg_net).

Diagrama de casos de uso (representación textual):

```
                    SISTEMA AGROGESTIÓN
┌──────────────────────────────────────────────────┐
│                                                  │
│  ┌──────────────┐     ┌──────────────────────┐   │
│  │  Registrarse  │     │  Iniciar sesión      │   │
```

```
│  └──────┬──────┘     └──────┬──────┘  │
│         │                    │         │
│  ┌──────▼────────────────────▼────────┐
│  │        Ver panel según rol         │
│  └──┬──────┬──────┬──────┬──────┬────┘
│     │      │      │      │      │
│  ┌──▼──┐ ┌──▼──┐ ┌──▼──┐ ┌──▼──────┐
│  │Admin│ │Gerente│ │Capataz│ │Trabajador│
│  │Panel│ │Panel │ │Panel │ │Panel    │
│  └─┬───┘ └─┬───┘ └─┬───┘ └─┬───────┘
│    │       │       │       │
│ Gestión  Terrenos  Tomar   Ver mis
│ usuarios Tareas    tareas  tareas
│ y roles  Capataces Completar
│ Editar   Mi equipo Asignar Editar
│ usuarios           trabajad. perfil
│ Estadíst. Editar
│           perfil  Editar perfil
│
│  ┌──────────────────────────────────┐
│  │    Cambiar idioma (ES / EN / RO) │
│  └──────────────────────────────────┘
│
│  ┌──────────────────────────────────┐
│  │          Cerrar sesión           │
│  └──────────────────────────────────┘
└──────────────────────────────────────┘
```

3.2. Desarrollo (Diagramas de secuencia)

3.2.1. Diagrama de secuencia: Inicio de sesión

```
Usuario          Frontend (React)        Supabase Auth         Base de datos
  │                   │                       │                      │
  │  Introduce email  │                       │                      │
  │  y contraseña     │                       │                      │
  │──────────────────►│                       │                      │
  │                   │                       │                      │
  │                   │  signInWithPassword() │                      │
  │                   │──────────────────────►│                      │
  │                   │                       │                      │
  │                   │                       │ Verifica credenciales│
  │                   │                       │──────────────────────►│
  │                   │                       │                      │
  │                   │                       │  ◄── Usuario válido  │
  │                   │                       │                      │
  │                   │ ◄── Token JWT + datos │                      │
  │                   │                       │                      │
  │                   │  Consulta rol usuario │                      │
  │                   │──────────────────────────────────────────────►│
  │                   │                       │                      │
  │                   │  ◄── Datos de rol     │                      │
```

```
  |                    |                      |                      |
  |  Redirige al panel |                      |                      |
  |  según rol         |                      |                      |
  |◄──────────────────|                      |                      |
```

3.2.2. Diagrama de secuencia: Registro de usuario

```
Visitante        Frontend (React)        Supabase Auth         Base de datos
  |                    |                      |                      |
  |  Rellena formulario|                      |                      |
  |  de registro       |                      |                      |
  |──────────────────►|                      |                      |
  |                    |                      |                      |
  |                    |  signUp(email, pass) |                      |
  |                    |──────────────────────►|                      |
  |                    |                      |                      |
  |                    |                      |  Crea usuario en Auth|
  |                    |                      |──────────────────────►|
  |                    |                      |                      |
  |                    |  ◄── Usuario creado  |                      |
  |                    |                      |                      |
  |                    |  Trigger: INSERT en  |                      |
  |                    |  tabla perfiles      |                      |
  |                    |                      |──────────────────────►|
  |                    |                      |                      |
  |                    |  ◄── Perfil creado   |                      |
  |                    |                      |                      |
  |  Mensaje de éxito  |                      |                      |
  |◄──────────────────|                      |                      |
```

3.2.3. Diagrama de secuencia: Crear tarea (Gerente)

```
Gerente          Frontend (React)          Supabase API          Base de datos
  |                    |                      |                      |
  |  Rellena formulario|                      |                      |
  |  de nueva tarea    |                      |                      |
  |──────────────────►|                      |                      |
  |                    |                      |                      |
  |                    |  Valida datos locales |                      |
  |                    |                      |                      |
  |                    |  INSERT tarea        |                      |
  |                    |  (terreno, capataz,  |                      |
  |                    |   estado: pendiente) |                      |
  |                    |──────────────────────►|                      |
  |                    |                      |                      |
  |                    |                      |  Verifica RLS        |
  |                    |                      |  (es gerente?)       |
  |                    |                      |──────────────────────►|
  |                    |                      |                      |
  |                    |                      |  ◄── INSERT OK       |
  |                    |                      |                      |
```

```
  |                    |  ←── Tarea creada     |                      |
  |                    |                       |                      |
  |  Actualiza listado |                       |                      |
  |  de tareas         |                       |                      |
  |←───────────────────|                       |                      |
```

3.2.4. Diagrama de secuencia: Cambiar estado de tarea (Capataz)

```
Capataz          Frontend (React)          Supabase API          Base de datos
  |                    |                       |                      |
  |  Pulsa "Tomar      |                       |                      |
  |  tarea" o          |                       |                      |
  |  "Completar"       |                       |                      |
  |───────────────────→|                       |                      |
  |                    |                       |                      |
  |                    |  UPDATE tarea         |                      |
  |                    |  SET id_estado = X    |                      |
  |                    |──────────────────────→|                      |
  |                    |                       |                      |
  |                    |                       |  Verifica RLS        |
  |                    |                       |  (es capataz de      |
  |                    |                       |   esta tarea?)       |
  |                    |                       |──────────────────────→|
  |                    |                       |                      |
  |                    |                       |  ←── UPDATE OK       |
  |                    |                       |                      |
  |                    | ←── Estado actualizado|                      |
  |                    |                       |                      |
  |  Actualiza vista   |                       |                      |
  |←───────────────────|                       |                      |
```

3.3. Pruebas realizadas

3.3.1. Pruebas funcionales

ID	Prueba	Resultado esperado	Estado
PF-01	Registro con datos válidos	Se crea el usuario y se muestra mensaje de éxito.	✅ OK
PF-02	Registro con email ya existente	Se muestra error indicando que el email ya existe.	✅ OK
PF-03	Login con credenciales correctas	Se redirige al panel correspondiente al rol.	✅ OK
PF-04	Login con credenciales incorrectas	Se muestra mensaje de error.	✅ OK
PF-05	Cerrar sesión	Se cierra la sesión y se redirige a la landing.	✅ OK

PF-06	Admin: ver lista de usuarios	Se muestra la tabla con todos los usuarios.	✅ OK
PF-07	Admin: asignar rol a usuario	El rol se actualiza en la base de datos.	✅ OK
PF-08	Gerente: crear terreno	El terreno aparece en el listado del gerente.	✅ OK
PF-09	Gerente: baja lógica de terreno	El terreno se marca como inactivo.	✅ OK
PF-10	Gerente: crear tarea	La tarea se asocia al terreno y capataz.	✅ OK
PF-11	Capataz: tomar tarea	El estado cambia a "en progreso".	✅ OK
PF-12	Capataz: completar tarea	El estado cambia a "completada".	✅ OK
PF-13	Capataz: asignar trabajador a tarea	El trabajador aparece asociado a la tarea.	✅ OK
PF-14	Trabajador: ver tareas asignadas	Se muestran solo las tareas del trabajador.	✅ OK
PF-15	Acceso no autorizado a datos de otro rol	Supabase RLS bloquea el acceso.	✅ OK
PF-16	Admin: editar datos de usuario	Se abre modal, se guardan cambios y la tabla se actualiza.	✅ OK
PF-17	Editar perfil personal	El usuario modifica nombre, apellidos, tlf, dirección.	✅ OK
PF-18	Recuperar contraseña	Se envía email de restablecimiento y se actualiza.	✅ OK
PF-19	Registro con selección de rol	El usuario se registra eligiendo Gerente, Capataz o Trabajador.	✅ OK
PF-20	Añadir comentario en tarea	El comentario aparece visible para los participantes de la tarea.	✅ OK
PF-21	Notificación automática al asignar tarea	El capataz recibe notificación al ser asignado a una tarea.	✅ OK
PF-22	Trabajador acepta/rechaza asignación	El estado de la asignación cambia a ACEPTADA o RECHAZADA.	✅ OK

3.3.2. Pruebas de interfaz / Responsive

ID	Prueba	Navegador/Dispositivo	Estado

PI-01	Visualización en escritorio (1920x1080)	Chrome	✅ OK
PI-02	Visualización en tablet (768x1024)	Firefox	✅ OK
PI-03	Visualización en móvil (375x667)	Chrome (DevTools)	✅ OK
PI-04	Navegación con menú hamburguesa en móvil	Chrome (DevTools)	✅ OK
PI-05	Formularios responsive	Edge	✅ OK
PI-06	Menú hamburguesa y overlay en móvil	Chrome (DevTools)	✅ OK
PI-07	Cambio de idioma (ES / EN / RO)	Chrome, Firefox	✅ OK
PI-08	Tarjetas de estadísticas coloreadas (Admin)	Chrome	✅ OK
PI-09	Gráfico de distribución de roles (Admin)	Chrome	✅ OK

3.3.3. Pruebas de seguridad

ID	Prueba	Resultado esperado	Estado
PS-01	Acceso directo a URL protegida sin sesión	Redirige al login.	✅ OK
PS-02	Inyección SQL vía formularios	Supabase sanitiza las consultas automáticamente.	✅ OK
PS-03	Acceso a datos de otro usuario vía API	RLS bloquea el acceso.	✅ OK

---

4. Proceso de despliegue

4.1. Requisitos de sistema

Para desarrollo local:

Requisito	Versión mínima
Node.js	18.x o superior (recomendado 20.x LTS)
npm	9.x o superior (incluido con Node.js)
Git	2.x o superior
Navegador web	Chrome 90+, Firefox 90+, Edge 90+, Safari 15+
Sistema operativo	Windows 10+, macOS 12+, Linux (Ubuntu 20.04+)

Para producción:

Requisito	Detalle
Cuenta en Supabase	Plan gratuito o de pago.

Plataforma de hosting	Vercel, Netlify u otro hosting estático.
Dominio (opcional)	Dominio personalizado si se desea.

4.2. Configuración de Supabase

1. Acceder a https://supabase.com y crear una cuenta.
2. Crear un nuevo proyecto con nombre "agrogestion".
3. En el panel de Supabase, acceder al **SQL Editor** y ejecutar los scripts de creación de tablas.
4. Configurar las **políticas RLS** para cada tabla según el rol del usuario.
5. En **Settings** → **API**, copiar la `URL del proyecto` y la `anon key`.
6. Crear el archivo `.env` en la raíz del proyecto frontend con:

```
VITE_SUPABASE_URL=https://tu-proyecto.supabase.co
VITE_SUPABASE_ANON_KEY=tu_clave_anon_publica
```

4.3. Subida de la aplicación a producción

Opción A: Despliegue en Vercel

1. Crear una cuenta en https://vercel.com.
2. Conectar el repositorio de GitHub (`Juanfrina/Agrogestion.Web`).
3. En **Settings** → **General**, configurar:
   - **Framework Preset:** Vite
   - **Root Directory:** `AGROGESTION` (la aplicación no está en la raíz del repositorio)
   - **Build Command:** `npm run build`
   - **Output Directory:** `dist`
4. Configurar las variables de entorno en **Settings** → **Environment Variables**:
   - `VITE_SUPABASE_URL`
   - `VITE_SUPABASE_ANON_KEY`
5. El proyecto incluye un archivo `vercel.json` con la configuración de reescritura SPA:

   ```
   { "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
   ```

   Esto garantiza que las rutas del lado del cliente (React Router) funcionen correctamente.
6. Vercel detectará automáticamente que es un proyecto Vite y lo construirá.
7. La URL de producción estará disponible tras el despliegue.

Opción B: Despliegue en Netlify

1. Crear una cuenta en https://netlify.com.
2. Importar el repositorio de GitHub.
3. Configurar:
   - **Build command:** `npm run build`
   - **Publish directory:** `dist`
4. Añadir variables de entorno.
5. Publicar.

Opción C: Despliegue manual (servidor propio)

1. Ejecutar `npm run build` para generar la carpeta `dist/`.
2. Subir el contenido de `dist/` al servidor web (Apache, Nginx, etc.).

3. Configurar el servidor para servir una SPA (redirigir todas las rutas a `index.html`).

---

5. Propuestas de mejoras

5.1. Mejoras aplicadas durante el desarrollo

Mejora	Descripción
Migración a TypeScript	Se migró de JavaScript a TypeScript para mayor seguridad de tipos.
Uso de SWC en lugar de Babel	Compilación más rápida en desarrollo usando el plugin SWC.
Estructura modular del proyecto	Separación en carpetas: pages, components, context, hooks, router, database, interfaces, store, utils, lib.
TailwindCSS v4	Uso de la última versión de Tailwind con mejor rendimiento.
Variables de entorno con Vite	Uso de `import.meta.env` para gestionar variables sensibles.
Patrón Repository	Capa de abstracción para acceso a datos con implementaciones Supabase.
Internacionalización (i18n)	Soporte multi-idioma (español, inglés y rumano) con i18next y react-i18next.
Recuperación de contraseña	Flujo de restablecimiento de contraseña con Supabase Auth.
Store de estado global (Zustand)	Gestión centralizada del estado de autenticación con Zustand.
Gráficos mensuales	Componente `MensualChart` para visualización de datos con gráficos de barras mensuales.
Diseño responsive con hamburguesa	Sidebar colapsable con menú hamburguesa, overlay en móvil y adaptación completa a pantallas pequeñas.
Edición de usuarios por Admin	Modal de edición con campos nombre, apellidos, email, teléfono, dirección y rol; con validaciones.
Edición de perfil personal	Cualquier usuario puede editar su nombre, apellidos, teléfono y dirección desde "Mi Perfil".
Tarjetas de estadísticas coloreadas	Panel de Admin con tarjetas (usuarios, terrenos, tareas, roles) con colores diferenciados por categoría.
Gráfico de distribución de roles	Componente `RoleDistributionChart` que muestra la cantidad de usuarios por rol con colores asignados.
Selección de rol en registro	El usuario elige su rol (Gerente, Capataz, Trabajador) al registrarse, sin intervención del administrador.
Comentarios en tareas	Tabla `comentarios_tarea` con políticas RLS para que solo participantes de la tarea puedan comentar.
Notificaciones automáticas	Triggers en PostgreSQL que generan notificaciones al asignar tareas o trabajadores.

Notificaciones por email	Trigger `enviar_email_notificacion()` que envía emails vía Brevo (API REST con `pg_net`) al crear una notificación.
Flujo de aceptación de trabajadores	Los trabajadores pueden aceptar o rechazar su asignación a una tarea (estado en `tarea_trabajador`).

5.2. Mejoras propuestas para futuras versiones

Mejora	Descripción
Notificaciones en tiempo real	Usar Supabase Realtime para notificar cambios de estado en tareas sin polling.
Exportación de informes en PDF	Generar informes de tareas y terrenos en formato PDF.
Geolocalización de terrenos	Integrar un mapa interactivo (Leaflet/Google Maps) para ubicar terrenos.
Sistema de comentarios en tareas	Permitir que capataces y trabajadores añadan comentarios.
Dashboard avanzado con filtros	Ampliar estadísticas con filtros por fecha, terreno y rol.
Modo offline con Service Worker	Cachear datos para poder consultar información sin conexión.
Tests automatizados	Implementar testing con Vitest y React Testing Library.
Carga de imágenes	Subir fotos de terrenos o tareas usando Supabase Storage.
App móvil nativa	Versión con React Native o PWA para móviles.

---

6. Bibliografía

Recurso	URL
React – Documentación oficial	https://react.dev
Vite – Documentación oficial	https://vite.dev
TypeScript – Documentación oficial	https://www.typescriptlang.org/docs/
TailwindCSS – Documentación oficial	https://tailwindcss.com/docs
Supabase – Documentación oficial	https://supabase.com/docs
Supabase Auth – Guía de autenticación	https://supabase.com/docs/guides/auth
Supabase RLS – Row Level Security	https://supabase.com/docs/guides/auth/row-level-security
@supabase/supabase-js – Librería cliente	https://github.com/supabase/supabase-js
React Router – Documentación	https://reactrouter.com
ESLint – Documentación oficial	https://eslint.org

Node.js – Documentación oficial	https://nodejs.org
npm – Documentación oficial	https://docs.npmjs.com
MDN Web Docs – Referencia web	https://developer.mozilla.org
i18next – Framework de internacionalización	https://www.i18next.com
react-i18next – Integración React	https://react.i18next.com
Zustand – Gestión de estado	https://zustand-demo.pmnd.rs
Vercel – Plataforma de despliegue	https://vercel.com/docs
Netlify – Plataforma de despliegue	https://docs.netlify.com